TECHNICAL CURRICULUM & SPECIFICATION

Production Agentic AI: Zero to Enterprise Deployment

A GitHub-First Engineering Course for Deterministic Control, Stateful Graphs, and Resilient Operations

1. Course Overview & Design Philosophy

Most educational resources teach toy agents. Toy agents use unconstrained model loops, store memory in local variables, and fail in production environments. This curriculum teaches software engineers how to build, test, secure, and deploy enterprise-grade agentic systems. Every module pairs architectural theory with executable code and automated verification gates.

> **Core Engineering Principles:**
>
> 1. **First Principles First:** Build core loops in standard Python before adopting orchestration frameworks.
> 2. **Deterministic Boundaries:** Constrain stochastic model calls with strict schemas, state machines, and budget caps.
> 3. **Zero-Cost Local Testing:** Use mock inference servers and reproducible containers to test logic without API expenses.
> 4. **Continuous Verification:** Every module lab includes failing `pytest` test suites that students must make pass.

2. Master Technology & Package Matrix

The table below lists the required packages, frameworks, and infrastructure tools for each module in the course.

Module	Core Focus	Primary Packages & Frameworks	Infrastructure & CLI
Module 0	Environment & Mock Testing	`pytest`, `pytest-asyncio`, `httpx`, `respx`, `fastapi`, `uvicorn`	`uv`, Docker Compose, DevContainers
Module 1	Deterministic Control Loops	`openai` / `anthropic` SDKs, `pydantic`, `tiktoken`, `tenacity`	Python 3.12+ Standard Library
Module 2	MCP Tools & Sandboxing	`fastmcp`, `mcp`, `psycopg[binary]`, `sqlalchemy`, `docker` (SDK)	PostgreSQL 16, Ephemeral Docker Containers
Module 3	Context & Durable State	`langgraph-checkpoint-postgres`, `pgvector`, `asyncpg`, `tiktoken`	PostgreSQL + `pgvector` Extension

Module	Core Focus	Primary Packages & Frameworks	Infrastructure & CLI
Module 4	Graph Orchestration & Routing	`langgraph`, `langchain-core`, `litellm`, `pydantic-settings`	Local Quantized Models (Ollama) + Frontier APIs
Module 5	Governance, Guardrails & HITL	`presidio-analyzer`, `presidio-anonymizer`, `redis`, `fastapi`	Redis 7 (Idempotency & Locks)
Module 6	Telemetry & Trajectory Evals	`opentelemetry-api`, `opentelemetry-sdk`, `arize-phoenix`, `deepeval`	OpenTelemetry Collector, GitHub Actions
Module 7	Async Queues & Fault Tolerance	`celery` / `arq`, `sse-starlette`, `redis`, `tenacity`	Redis Streams / PubSub, Background Workers
Module 8	Capstone Enterprise System	Full Stack Integration (`fastapi`, `langgraph`, `fastmcp`, `celery`, `otel`)	Multi-Container Docker Compose, Nginx

3. GitHub Repository Structure

Structure the course repository into three learning phases. Each module contains three subfolders: `theory/` for system design documents, `lab/` for student starter code with failing tests, and `solution/` for completed reference implementations.

```
production-agentic-ai-course/
├── .devcontainer/                  # VS Code DevContainer configuration
├── .github/workflows/ci.yml        # Automated trajectory and unit test runner
├── docker-compose.yml              # PostgreSQL (pgvector), Redis, Phoenix, Mock
LLM
├── pyproject.toml                  # Workspace dependencies locked with uv
├── README.md                       # Course syllabus, setup rules, and
architecture map
├── phase-1-foundations/
│   ├── 00-environment-and-mocks/
│   ├── 01-deterministic-control-flow/
│   └── 02-mcp-tools-and-sandboxing/
├── phase-2-state-and-orchestration/
│   ├── 03-context-and-durable-state/
│   ├── 04-graph-orchestration-and-routing/
│   └── 05-governance-guardrails-hitl/
└── phase-3-enterprise-operations/
    ├── 06-telemetry-and-trajectory-evals/
    ├── 07-async-queues-and-resilience/
    └── 08-capstone-multitenant-deployment/
```

4. Phase 1: Agentic Foundations and Boundaries

Phase 1 strips away high-level framework abstractions. Students build core reasoning loops in standard Python, validate outputs with strict schemas, and isolate tool execution using the Model Context Protocol (MCP).

Module 0: Architecture Foundations and Reproducible Environments

> **Tech Stack & Packages:** `uv`, `docker-compose`, `pytest`, `pytest-asyncio`, `httpx`, `respx`, `fastapi`, `uvicorn`

- **Failure Mode Addressed:** Dependency conflicts, unrepeatable local setups, and high API bills during early testing.
- **Theory Topics (`theory/README.md`):**
  - Why prototype agents fail in production: compounding latency, non-deterministic loops, and state loss.
  - The Agency Spectrum: when to write deterministic procedural code instead of calling an LLM.
  - Deterministic replay testing: recording and mocking LLM HTTP responses for fast unit tests.
- **Hands-On Lab (`lab/`):**
  - Initialize the Python workspace with `uv` and boot PostgreSQL and Redis via Docker Compose.
  - Build a local FastAPI Mock LLM Server that returns pre-recorded tool calls and simulates network latency.
  - Write `respx` network interceptors to block accidental external API calls during unit test runs.
- **Production Verification Gate (`pytest`):** Run 50 simulated multi-turn conversations through the test harness in under 2 seconds with zero external network traffic.

Module 1: Deterministic Control Flow and Core Reasoning Loops

> **Tech Stack & Packages:** `openai`, `anthropic`, `pydantic` (≥2.9), `tiktoken`, `tenacity`

- **Failure Mode Addressed:** Infinite execution loops, token budget exhaustion, malformed JSON errors, and prompt injection via user data.
- **Theory Topics (`theory/README.md`):**
  - Anatomy of a bounded ReAct (Reason + Act) state loop without third-party agent libraries.
  - Writing system prompts as strict operating manuals: role definitions, hard constraints, output contracts, and failure states.
  - Data vs. Instruction separation: wrapping untrusted external inputs in XML tags (e.g., `<user_data>`) to prevent indirect prompt injection.
  - Self-correction loops: feeding Pydantic `ValidationError` traces back to the model for automatic repair.

- **Hands-On Lab (`lab/`):**
  - Build a pure Python `AgentRuntime` class with hard step ceilings (`max_steps=5`) and real-time token counters using `tiktoken`.
  - Enforce structured tool call outputs with Pydantic models and implement a 2-attempt schema repair loop.
  - Implement hard cost budget limits that raise a custom `BudgetExceededError` before exceeding session thresholds.
- **Production Verification Gate (`pytest`):** Pass 20 adversarial test inputs (including malformed JSON responses, infinite loop traps, and prompt injection payloads) without unhandled exceptions or step ceiling violations.

Module 2: Tool Sandboxing, FastMCP, and Schema Governance

> **Tech Stack & Packages:** `fastmcp`, `mcp`, `psycopg[binary]`, `sqlalchemy`, `docker` (Python SDK), `pydantic`

- **Failure Mode Addressed:** Tool poisoning, SQL injection, arbitrary code execution on host machines, credential leaks in prompts, and breaking schema changes.
- **Theory Topics (`theory/README.md`):**
  - Model Context Protocol (MCP) architecture: hosts, clients, servers, and transport mechanisms (stdio vs. Streamable HTTP).
  - Security boundaries: separating read-only query tools from state-mutating write tools.
  - Zero-trust secret injection: passing database credentials and API keys at the MCP server layer so secrets never enter the LLM context window.
  - Tool schema versioning: maintaining v1 and v2 tool signatures simultaneously so active checkpoints do not break during deployments.
- **Hands-On Lab (`lab/`):**
  - Build a custom `FastMCP` server that exposes parameterized PostgreSQL query tools with strict Pydantic input validation.
  - Build an isolated code-execution tool using the Python `docker` SDK that runs agent-generated scripts inside a network-disabled container with a 64MB RAM cap and read-only root filesystem.
  - Implement a schema version adapter that upgrades deprecated v1 tool calls from older sessions into v2 schema formats automatically.
- **Production Verification Gate (`pytest`):** Execute a red-team test suite that attempts SQL injection, host file access, and outbound network requests from inside the tool sandbox; verify 100% of attacks are blocked.

5. Phase 2: State, Orchestration, and Governance

Phase 2 transitions from single-loop scripts to durable state machines. Students learn how to manage long-running conversations, persist state across server restarts, optimize model costs, and enforce human approval gates.

Module 3: Context Engineering, Compaction, and Durable State

> **Tech Stack & Packages:** `langgraph-checkpoint-postgres`, `pgvector`, `asyncpg`, `psycopg_pool`, `tiktoken`

- **Failure Mode Addressed:** Context window overflow, instruction drift in long sessions, and lost execution progress when worker processes crash.
- **Theory Topics (`theory/README.md`):**
  - Context degradation ("lost in the middle" phenomenon) and why raw append-only chat history fails in production.
  - Context compaction strategies: sliding window trimming, recursive background summarization, and pruning verbose tool payloads after consumption.
  - Separating volatile conversational memory from structured execution state.
  - Write-ahead logging (WAL) and transactional checkpointing: saving state snapshots after every state transition.
- **Hands-On Lab (`lab/`):**
  - Configure a PostgreSQL checkpoint store using `AsyncPostgresSaver` and connection pooling.
  - Build a `ContextCompactor` middleware that keeps the system operating manual and last 4 turns intact while summarizing older messages into a structured state object.
  - Implement time-travel debugging: load an historical checkpoint ID, modify a corrupt tool response in the state dictionary, and fork execution from that exact point.
- **Production Verification Gate (`pytest`):** Terminate the agent worker process mid-run with `SIGKILL`, restart the process, reload the session from PostgreSQL, and verify the task finishes without repeating earlier tool calls.

Module 4: Graph Orchestration, Multi-Model Routing, and Effort Control

> **Tech Stack & Packages:** `langgraph`, `langchain-core`, `litellm`, `pydantic-settings`

- **Failure Mode Addressed:** High latency and runaway API costs caused by using expensive frontier models for trivial routing or formatting steps.
- **Theory Topics (`theory/README.md`):**
  - Production graph topologies: Router-Specialist, Orchestrator-Worker, and Evaluator-Optimizer (Generator-Verifier).
  - Why autonomous peer-to-peer multi-agent swarms are an anti-pattern in enterprise production.
  - Multi-model routing and Reasoning Effort Dials: using high-reasoning frontier models for initial planning and fast, low-cost models for worker execution.
  - Conditional verification: triggering expensive Evaluator nodes only when confidence scores are low or when state mutations occur.
  - Prompt caching mechanics: structuring static prefixes (system prompts, tool schemas) to achieve 80%+ cache hit rates.

- **Hands-On Lab (`lab/`):**
  - Refactor the Module 1 Python loop into a strictly typed `StateGraph` in LangGraph.
  - Configure `LiteLLM` to route intent classification and JSON formatting nodes to a fast small model, while routing complex diagnostics to a primary reasoning model.
  - Build an Evaluator-Optimizer cycle where a deterministic code linter checks agent output before allowing the graph to reach the `END` node.
- **Production Verification Gate (`pytest`):** Run a 25-task benchmark suite and verify at least a 45% reduction in total token cost and 30% lower latency compared to a single-frontier-model baseline, with zero drop in accuracy.

Module 5: Governance, Guardrails, Idempotency, and Human-in-the-Loop (HITL)

> **Tech Stack & Packages:** `langgraph`, `presidio-analyzer`, `presidio-anonymizer`, `redis`, `fastapi`

- **Failure Mode Addressed:** Duplicate database writes or refunds during network retries, unauthorized destructive actions, and PII data leaks to external model providers.
- **Theory Topics (`theory/README.md`):**
  - Idempotency in agentic workflows: generating deterministic idempotency keys (`hash(session_id + step_id + tool_args)`) so retried tool calls execute exactly once.
  - Human-in-the-Loop (HITL) state suspension: using graph interrupts to pause execution indefinitely without holding open threads.
  - Data governance and privacy: scrubbing PII (names, credit cards, SSNs) with local token vaults before sending prompts to external APIs, and re-hydrating tokens locally.
  - Role-Based Access Control (RBAC) for tools and automated circuit breakers that trip when error rates spike.
- **Hands-On Lab (`lab/`):**
  - Build a Redis-backed `@idempotent_tool` decorator that caches and returns previous execution results if an identical write request arrives twice.
  - Configure Microsoft Presidio middleware to redact sensitive customer fields before model inference.
  - Add a LangGraph `interrupt_before=["execute_database_patch"]` gate and build a FastAPI webhook where a human reviewer can approve, edit parameters, or reject the action.
- **Production Verification Gate (`pytest`):** Simulate three network timeouts during a payment refund tool call and confirm the database records exactly one refund; verify that unapproved destructive actions remain suspended in PostgreSQL.

6. Phase 3: Enterprise Operations (AgentOps)

Phase 3 prepares the system for live production traffic. Students implement distributed tracing, automated CI/CD trajectory evaluations, background worker queues, provider failover, and multi-tenant isolation.

Module 6: Telemetry, Observability, and Trajectory Evaluations

> **Tech Stack & Packages:** `opentelemetry-api`, `opentelemetry-sdk`, `openinference-instrumentation-langchain`, `arize-phoenix`, `deepeval`

- **Failure Mode Addressed:** Silent regressions after prompt edits, inability to debug multi-step failures, and unmeasured token latency bottlenecks.
- **Theory Topics (`theory/README.md`):**
  - Distributed tracing for agents: instrumenting traces, spans, prompt/completion token counts, cache hit ratios, and tool latency using OpenTelemetry.
  - The Three Pillars of Agent Evaluation:
    1. Deterministic Unit Assertions (exact tool name match, Pydantic argument validity).
    2. Trajectory Evaluation (comparing actual tool sequences against golden reference paths; measuring step efficiency).
    3. Calibrated LLM-as-a-Judge (grading final response groundedness and tone against strict rubrics).
  - Continuous Integration (CI) gates: blocking pull requests when trajectory precision degrades.
- **Hands-On Lab (`lab/`):**
  - Instrument the LangGraph workflow and FastMCP server with OpenTelemetry spans and export traces to a local Arize Phoenix container.
  - Curate a Golden Benchmark Dataset of 30 realistic multi-turn tickets (JSONL format) with expected tool trajectories.
  - Write a `pytest` + `deepeval` test suite and configure a GitHub Actions workflow that scores trajectory recall, tool precision, and step overhead on every commit.
- **Production Verification Gate (`pytest`):** Introduce a deliberate prompt regression into the router node; verify the CI evaluation script detects the suboptimal tool path and fails the build with a clear diff report.

Module 7: Asynchronous Workloads, Task Queues, and Fault Tolerance

> **Tech Stack & Packages:** `celery` (or `arq`), `redis`, `fastapi`, `sse-starlette`, `tenacity`, `litellm`

- **Failure Mode Addressed:** HTTP gateway timeouts during slow multi-step agent runs, dropped client connections, and service outages when a primary LLM provider goes down.
- **Theory Topics (`theory/README.md`):**
  - Decoupling HTTP web servers from agent execution: enqueueing runs into distributed task workers (Celery / Redis Streams).
  - Real-time state streaming: publishing intermediate node transitions and token chunks over Redis Pub/Sub to FastAPI Server-Sent Events (SSE) endpoints.
  - Resilience engineering: exponential backoff with jitter, respecting `Retry-After` headers, dead-letter queues (DLQ) for poisoned tasks, and multi-provider failover (e.g., OpenAI → Anthropic → Local Fallback).

- ◦ Graceful container shutdowns: handling `SIGTERM` signals so workers finish saving their current checkpoint before exiting.
- **Hands-On Lab (`lab/`):**
  - ◦ Move LangGraph execution out of the FastAPI request thread into an asynchronous Redis worker pool.
  - ◦ Build an SSE streaming endpoint (`GET /v1/runs/{run_id}/stream`) with cursor resumption (`Last-Event-ID`) so disconnected clients can reconnect without losing tokens.
  - ◦ Configure a provider failover chain that automatically switches to a secondary model API when the primary API returns `HTTP 429` or `HTTP 503` .
- **Production Verification Gate (`pytest`):** Inject `HTTP 503` errors into the primary model mock while firing 25 concurrent client requests; verify 100% of runs complete via the backup provider with zero HTTP timeouts.

Module 8: Capstone — Multi-Tenant Enterprise Deployment

> **Tech Stack & Packages:** Full Stack (`fastapi` , `langgraph` , `fastmcp` , `psycopg` , `pgvector` , `redis` , `celery` , `opentelemetry` , Docker Compose)

- **Capstone System Specification:** Build, secure, and deploy an **Autonomous Enterprise Incident Triage & Remediation Service**. The system ingests production alerts, searches runbooks in `pgvector` , queries live telemetry databases via FastMCP, diagnoses root causes, proposes remediation scripts, pauses for human approval before applying fixes, and logs complete OpenTelemetry audit trails.
- **Theory Topics (`theory/README.md`):**
  - ◦ Multi-tenant isolation: enforcing strict `tenant_id` boundaries across PostgreSQL checkpoints, Redis keys, and `pgvector` embeddings using PostgreSQL Row-Level Security (RLS).
  - ◦ Shadow deployments and canary testing: duplicating a percentage of live production traffic asynchronously to a candidate agent version and comparing trajectory diffs before cutover.
  - ◦ Production readiness checklist: container hardening, non-root Docker users, liveness/readiness probes, and per-tenant rate limiting.
- **Hands-On Lab (`lab/`):**
  - ◦ Enable PostgreSQL Row-Level Security (RLS) policies on all checkpoint and vector tables so Tenant A can never query Tenant B's state or runbooks.
  - ◦ Build an asynchronous Shadow Router in FastAPI that mirrors live requests to a `v2-canary` graph worker and logs trajectory comparison metrics.
  - ◦ Assemble the production `docker-compose.prod.yml` stack with health checks, resource limits, and automated startup migrations.

- **Final Production Verification Gate (`pytest`):** Pass the full Capstone Readiness Suite:
  1. Cross-tenant penetration test confirms zero data leakage between tenants.
  2. Idempotency and HITL gates prevent unauthorized or duplicate remediation writes.
  3. Shadow router logs side-by-side v1 vs. v2 trajectory metrics without adding latency to the primary user response.
  4. System maintains 100% uptime during simulated worker restarts and primary LLM outages.

7. Student Workflow for Every GitHub Module

To keep the learning experience structured and consistent across all nine modules, every student follows the same four-step engineering workflow:

1. **Read the Architecture Guide:** Study the state diagrams, failure mode analysis, and trade-off tables in `theory/README.md`.
2. **Run the Failing Test Suite:** Execute `uv run pytest lab/tests/` to observe the exact production failures (e.g., infinite loop, SQL injection, state loss, or schema mismatch).
3. **Implement the Production Controls:** Write the required control flow, MCP boundaries, checkpointers, or guardrails inside `lab/src/`.
4. **Verify against the Gate:** Re-run `uv run pytest lab/tests/` and inspect local OpenTelemetry traces in Arize Phoenix until all functional, security, and cost assertions pass.